

# UI, API and Data Test Strategy

How the three QA-owned layers are structured, how they align to user stories, how they run in the pipeline, and the six scenarios that genuinely still need a human.

|                                            |                                                  |                                              |                                       |
|--------------------------------------------|--------------------------------------------------|----------------------------------------------|---------------------------------------|
| AUTHOR<br>Shanthi Subramanian ·<br>BA / QA | DATE<br>17 September 2026                        | EXCLUDES<br>Unit testing — Dev<br>owned      | DAYS TO SUBMIT<br>13                  |
| <div>18</div> <div>AUTOMATED TODAY</div>   | <div>24</div> <div>DATA TESTS TO<br/>BUILD</div> | <div>6</div> <div>MANUAL<br/>SCENARIOS</div> | <div>0</div> <div>RUNNING IN CI</div> |

## Scope & ownership

What QA owns, what Dev owns, and why the boundary matters more than usual on this project.

| LAYER         | OWNER | PROVES                                      | TODAY             |
|---------------|-------|---------------------------------------------|-------------------|
| Unit          | DEV   | Service logic in isolation                  | 62 tests, passing |
| API           | QA    | Contract, authorization, error mapping      | 12 tests          |
| UI            | QA    | The screens compose into a working workflow | 6 tests           |
| Data          | QA    | Database-enforced governance invariants     | NOTHING           |
| AI evaluation | QA    | Model output quality                        | 2 of 12 executing |

### ONE CONSEQUENCE OF THE BOUNDARY WORTH NAMING

Unit testing being Dev-owned is the right call — it is their design, their refactoring safety net. But it is worth recording that **DEF-004, the most serious governance defect found so far, was found by a unit test**, then confirmed through the UI days later. With QA out of that layer, defects of that shape now depend on the Dev team writing tests for the negative cases of their own invariants.

This is not an argument to change the boundary. It is an argument for the **Data layer** below to be built properly, because it is the only remaining layer where QA can prove a governance rule independently of the implementation that is supposed to enforce it.

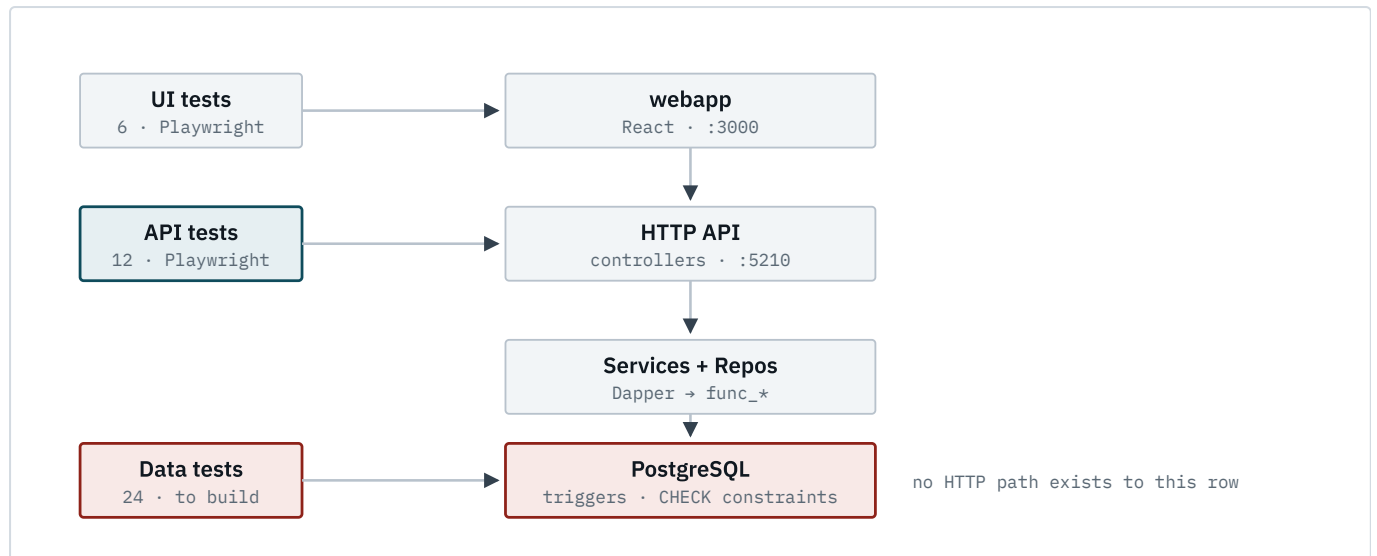
## What each layer must not be asked to do

The single most common way a test strategy fails is asking one layer to cover another's ground. The rule here:

- **API tests never assert business arithmetic** — that is the unit layer's job and it does it in milliseconds.
- **UI tests never re-prove API behaviour** — a browser is the most expensive way possible to check a status code.
- **Data tests never go through HTTP** — the whole point is reaching what HTTP cannot.

# The three layers

What each covers, with the reasoning for its size.



**Figure 1.** Each layer enters the stack at a different point. The bottom row is the one that matters: no endpoint anywhere attempts `UPDATE audit_event`, so no UI or API test can ever exercise the append-only trigger — only a direct connection can.

## API layer 12 TODAY → ~30 TARGET PLAYWRIGHT REQUEST

The workhorse. Fast, headless, no browser, and it is where authorization actually lives. Every new endpoint gets API coverage before it gets UI coverage.

### Covers

- **Contract** — the `OperationResult` envelope, status-code mapping, response shapes
- **Authorization** — role separation across the three roles on every write endpoint DEF-001/002
- **Validation** — mandatory reasons, mitigation-factor bounds, supported content types
- **Error handling** — unknown ids as 4xx not 500 DEF-003
- **Workflow sequencing** — a vote is refused before routing; finalize is refused before review
- **Concurrency** — two simultaneous workspace opens DEF-006

### Deliberately excluded

Scoring arithmetic and quorum resolution. Both are covered by unit tests in milliseconds; re-asserting them over HTTP buys nothing and slows every PR.

## UI layer

6 TODAY → ~12 TARGET

PLAYWRIGHT BROWSER

Deliberately the smallest suite. UI tests are the most expensive evidence per hour in this repo — they need Vite plus both services plus Postgres — and a broad suite would mostly re-prove what cheaper layers already cover.

### Covers

- **The three role journeys** — PO raises → Analyst assesses and finalizes → Committee votes. This single path *is* the product demo
- **Governance gates as a user meets them** — the Finalize tab refusing an unreviewed assessment  
DEF-004
- **Override dialogs** — that a reason is genuinely mandatory in the UI, not just server-side
- **Role-based navigation** — each role sees only its own screens
- **Intake validation** — type-specific fields appear and mandatory fields block submission

### Growth rule

A new UI test must justify why the API layer cannot prove the same thing. Twelve is the ceiling, not a target.

## Data layer 0 TODAY → 24 TARGET TESTCONTAINERS + NPGSQL

**The gap, and the highest-value work left.** This layer connects directly to a disposable PostgreSQL container with the real schema applied, and asserts the things the database itself enforces. It is the only place the product's flagship governance claims can be proven.

### Covers — governance invariants

| ID    | TEST                                                                       | STORY   |
|-------|----------------------------------------------------------------------------|---------|
| DT-01 | <code>UPDATE audit_event</code> is rejected by the trigger                 | US-9.2  |
| DT-02 | <code>DELETE audit_event</code> is rejected by the trigger                 | US-9.2  |
| DT-03 | No <code>func_*</code> routine exists that updates or deletes an audit row | US-9.2  |
| DT-04 | <code>residual_rating = 0</code> is rejected by CHECK                      | US-7.1  |
| DT-05 | <code>max_mitigation_factor = 1.0</code> is rejected by CHECK              | US-10.1 |
| DT-06 | An override with a null reason is rejected by CHECK                        | US-7.2  |
| DT-07 | Worst-case arithmetic still leaves residual above zero                     | US-7.1  |

### Covers — transactional integrity

| ID        | TEST                                                                                                                                                             | STORY             |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| DT-08..13 | Each state-changing <code>func_*</code> writes its <code>audit_event</code> row <b>in the same transaction</b> — override, correct, review, edit, finalize, vote | US-6.1, US-9.1    |
| DT-14     | A failed state change leaves no orphan audit row                                                                                                                 | US-9.1            |
| DT-15     | Concurrent <code>func_getOrCreateAssessment</code> calls yield exactly one assessment                                                                            | US-2.1<br>DEF-006 |

## Covers — data quality and reference integrity

| ID    | TEST                                                                          | STORY             |
|-------|-------------------------------------------------------------------------------|-------------------|
| DT-16 | Seed applies cleanly to an empty database and is re-runnable                  | US-12.1           |
| DT-17 | Four FFIEC categories with 21 sub-factors, each carrying a citation string    | US-2.1            |
| DT-18 | Every risk category has an active scoring config                              | US-7.1            |
| DT-19 | The control catalogue is non-empty per category                               | US-7.1<br>DEF-005 |
| DT-20 | Change-type → category mapping covers all six change types                    | US-2.1            |
| DT-21 | An external snapshot is immutable once written                                | US-9.2            |
| DT-22 | Replacing an attachment versions it rather than overwriting                   | US-1.2            |
| DT-23 | A config change creates a new version; in-flight assessments keep the old one | US-10.1           |
| DT-24 | A full audit chain for one request reconstructs in order                      | US-9.1            |

---

## Aligning tests to user stories

One convention, applied everywhere, which makes traceability a by-product rather than a separate document to maintain.

### The convention

Every test names the story and criterion it proves, in the test itself — not in a spreadsheet that drifts. Two forms, one per language:

```
// Playwright
test('US-8.2 AC5 – a vote is recorded against the member by name', ...)

// Testcontainers (xUnit)
[Fact(DisplayName = "US-9.2 – UPDATE on audit_event is rejected")]
```

Because Playwright and the .NET test runner both emit the test name into their result files, a traceability matrix can then be *generated* from a CI run rather than written by hand. A story with no matching test name shows up as an empty row automatically.

#### WHY THIS MATTERS MORE THAN IT SOUNDS

The current traceability matrix says 12 of 22 stories covered, 6 partial, 4 with nothing. That was assembled by hand and is already at risk of going stale. Once the naming convention is in place the matrix becomes a build artifact — always current, and impossible to quietly let rot.

### Coverage rule per story type

| STORY INVOLVES                                     | MINIMUM COVERAGE                                                                                          |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| An AI touchpoint (Epics 2, 4, 5)                   | API test for the guard + eval case for quality + UI test for the review gate                              |
| A human override with a mandatory reason           | API test that a blank reason is refused + Data test that the audit row is written in the same transaction |
| A governance invariant (residual > 0, append-only) | <b>Data test</b> — non-negotiable, an API test cannot reach it                                            |
| A role restriction                                 | API test per role, positive and negative                                                                  |
| A screen or a workflow transition                  | One UI journey step; no separate UI test                                                                  |
| A non-functional requirement                       | Usually pipeline configuration assertion, not a runtime test                                              |

---

## Automation design

Tooling per layer, and the patterns that keep the suites honest.

| LAYER     | TOOL                                                    | WHY THIS ONE                                                                                                                                   |
|-----------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| API       | Playwright <code>request</code>                         | Same tool as UI, so one install and one report. No browser needed, so it runs in seconds                                                       |
| UI        | Playwright browser                                      | Auto-waiting removes the sleep-and-hope pattern; trace viewer makes a CI failure diagnosable without reproducing locally                       |
| Data      | Testcontainers for PostgreSQL + Npgsql, driven by xUnit | The only way to reach triggers and constraints. Disposable container per run means no shared-state flakiness. Docker is already a prerequisite |
| Reporting | Playwright HTML + TRX to Azure DevOps                   | Native test-results rendering in the pipeline, with history                                                                                    |

### Four patterns carried across every suite

#### 1 · Arrange via API, assert via UI

A UI test whose subject is the committee stage should not re-drive intake through the browser. Create the precondition over HTTP in milliseconds, then drive only the part under test. Faster, and a failure points at the thing being tested rather than at setup.

#### 2 · Known defects are recorded, never deleted

Where the product does not yet do the right thing, the test asserts the *right* behaviour and is marked expected-to-fail. The board stays green, and the moment the fix lands the runner reports "expected to fail, but passed" — the fix announces itself. Deleting the test, or weakening it to match the bug, loses the requirement permanently.

#### 3 · Never let a dead environment look like a pass

An expected-to-fail test passes when it fails for *any* reason, including a refused connection. Every suite therefore starts by confirming its dependency is actually reachable and aborts loudly if not. Without this, a suite run against nothing produces a fully green board.

#### 4 · Verify the suite by breaking the code

Green means nothing until something has been broken on purpose. Before trusting a governance test, invert the rule it guards and confirm precisely that test fails — no more, no fewer — then revert. This has already been done for the finalization gate and will be repeated for each Data-layer invariant.

# Pipeline integration

Where each suite runs, what gates a merge, and what merely reports.

## STARTING POINT

**No application pipeline exists.** Both Azure DevOps pipelines are path-filtered to `iac/environments/dev/*`, so nothing runs `dotnet test`, Playwright, or `docker build`. This section describes what to build, not what to change.

| STAGE             | TRIGGER          | RUNS                                                        | GATE                       | BUDGET |
|-------------------|------------------|-------------------------------------------------------------|----------------------------|--------|
| 1 • Build         | Every PR         | <code>dotnet build</code> , <code>npm run build</code>      | HARD FAIL                  | ~2 min |
| 2 • Unit          | Every PR         | <code>dotnet test</code> (Dev-owned)                        | HARD FAIL                  | ~1 min |
| 3 • Data          | Every PR         | Testcontainers suite                                        | HARD FAIL                  | ~3 min |
| 4 • API           | Every PR         | API + Postgres service container, Playwright api project    | HARD FAIL                  | ~3 min |
| 5 • Image         | Merge to main    | <code>docker build</code> ×2, Trivy image scan, push to ACR | HARD FAIL on HIGH/CRITICAL | ~5 min |
| 6 • UI            | Merge to main    | Full stack in containers, Playwright ui project             | SOFT AT FIRST              | ~5 min |
| 7 • Evals         | Nightly + manual | AI evaluation harness against Foundry                       | REPORT ONLY                | ~3 min |
| 8 • Deploy verify | After deploy     | Schema/seed check, smoke against the deployed FQDN          | HARD FAIL                  | ~1 min |

Total PR feedback: **under 10 minutes**. That number is the design constraint — a PR gate slower than that gets bypassed, and a bypassed gate is worse than none because it creates false confidence.

## Why Data gates the PR but UI does not

The Data suite is fast, deterministic and guards the invariants the product is judged on — it belongs on every PR. The UI suite needs four processes and a browser, is inherently slower, and its failures are more often environmental than real. Running it on merge keeps PR feedback quick while still catching a broken journey before it reaches a demo. Once it has proven stable for a week, promote it to a hard gate.

## Sketch

```
trigger:
  branches: [main]
  paths:
    include: [src/*, tests/*, webapp/*] # mirrors the IaC pipeline's own pattern

stages:
- stage: Verify
  jobs:
  - job: BuildAndTest
    services:
      postgres: postgres16 # service container for the API + Data suites
    steps:
      - task: UseDotNet@2 # SDK 10.0.x
      - script: dotnet build
      - script: dotnet test tests/...UnitTests --logger trx
      - script: dotnet test tests/...IntegrationTests --logger trx # Data layer
      - script: |
          dotnet run --project src/1-API/... &
          cd tests/e2e && npm ci && npm run test:api
      - task: PublishTestResults@2
      - publish: tests/e2e/playwright-report
```

### REUSE WHAT ALREADY WORKS

The existing IaC pipeline already installs tooling, handles soft/hard failure cleanly, and runs a Claude review step that posts findings to the PR. Mirroring its structure — including that review step over test results and coverage — makes the application pipeline read as deliberate rather than bolted on, and evidences SDLC automation breadth across stages rather than in one.

## Manual validation

Yes — six scenarios genuinely need a human. Each is here because automation would produce a confident answer to the wrong question.

Everything deterministic should be automated, and most of this product is deterministic. But six areas resist it, and pretending otherwise would be the more dangerous choice.

| ID    | SCENARIO                                                                                                                      | WHY A HUMAN                                                                                                                                                                                                                                                                                       | WHEN                               |
|-------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| MV-01 | <b>AI narrative quality</b> — read a sample of drafted narratives across all four risk categories                             | The eval harness checks a narrative is well-formed and mentions its citation. It cannot judge whether the reasoning is <i>sound</i> , whether a citation is apposite rather than merely present, or whether an analyst would be embarrassed to hand it to a committee. That is a domain judgement | Once per prompt change             |
| MV-02 | <b>Examiner-readiness of the audit export</b> — read a full export end to end as if you were the examiner                     | US-9.1 AC3 asks for a format "suitable for handing to an examiner without additional formatting work". No assertion can evaluate that; only reading it can                                                                                                                                        | Once per export change             |
| MV-03 | <b>Exploratory testing of the override flows</b> — free-text reasons, very long input, mid-edit navigation, double submission | Override is first-class everywhere and the input is unconstrained text. Scripted tests only find what they were told to look for; this is where the unexpected lives                                                                                                                              | Timeboxed, twice before submission |
| MV-04 | <b>First deployment to Azure</b> — walk one full change request through the deployed environment                              | Automated smoke checks confirm services answer. Only a human walkthrough catches a broken redirect, a CORS origin missed, or an environment variable that is present but wrong                                                                                                                    | Once, at first deploy              |
| MV-05 | <b>Accessibility and usability spot-check</b> — keyboard-only traversal of the Assessment Workspace, contrast, focus order    | Automated scanners catch perhaps a third of real accessibility problems. The workspace is the screen an analyst lives in all day                                                                                                                                                                  | Once before submission             |
| MV-06 | <b>Committee decision semantics</b> — play out reject-with-rationale, defer, and approve-with-conditions with two real people | The quorum rule is unit-tested and correct. Whether the <i>workflow</i> makes sense to people deliberating — is the rationale visible to the second voter? does a defer read as a decision or a delay? — is a human question                                                                      | Once, with the team                |

### THE LINE I AM DRAWING

These six exist because **a test can only check what it was told to check**. MV-01 and MV-02 in particular are where an AI-assisted governance product either earns trust or quietly loses it — and both are exactly the kind of thing a green dashboard will never flag. Everything else in this strategy is automated, on purpose.

## **How manual results are recorded**

Each scenario gets a short written outcome in the test case register — date, who ran it, what they found — so that "we checked" is evidenced rather than remembered. A manual pass with no note is indistinguishable from a manual test that never ran.

---

## Entry & exit criteria

What must be true to start testing, and what must be true to submit.

### Entry

- The stack starts from a fresh clone using documented commands, on a machine with only Docker and the .NET SDK
- Schema and seed apply cleanly; `/api/Ping` returns 200
- The story under test has acceptance criteria written in Given/When/Then form

### Exit — the submission bar

| CRITERION                                                                                          | STATUS       |
|----------------------------------------------------------------------------------------------------|--------------|
| All four governance invariants proven by a Data-layer test that has been shown to fail when broken | NOT MET      |
| CI runs Build, Unit, Data and API on every PR                                                      | NOT MET      |
| All 12 eval cases executed against a real model                                                    | 2 OF 12      |
| The three role journeys pass against a deployed environment, not just locally                      | NOT MET      |
| Every open defect either fixed, or documented with a reproduction and an owner                     | MET          |
| All six manual scenarios executed with written outcomes                                            | 0 OF 6       |
| Traceability matrix generated from a CI run, no story silently uncovered                           | MANUAL TODAY |

# The 13-day plan

Sequenced so that each item unblocks the next.

| DAYS  | WORK                                                                                         | WHY THEN                                                                                           |
|-------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 1-3   | <b>Data layer</b> — Testcontainers project, DT-01 to DT-07 (the governance invariants) first | Highest value, needs no API key and no running app, and closes the only structural gap in coverage |
| 2-4   | <b>Application CI</b> — stages 1 to 4                                                        | Runs in parallel with the above; depends on <code>release/1.00</code> reaching <code>main</code>   |
| 4-6   | Data layer DT-08 to DT-24; API suite to ~30 tests                                            | Once the harness exists, cases are cheap to add                                                    |
| 5-7   | Evals against Foundry; expand to adversarial cases                                           | Needs the model configured — the one dependency outside QA's control                               |
| 7-9   | Image + UI stages in CI; container test cases executed                                       | After the PR gate is proven stable                                                                 |
| 9-11  | <b>Manual scenarios MV-01 to MV-06</b>                                                       | Needs a deployed environment and final prompts, so it cannot come earlier                          |
| 11-13 | Regenerate traceability from CI; final regression; documentation freeze                      | Everything else must be stable first                                                               |

THE TWO DEPENDENCIES OUTSIDE QA'S CONTROL

Days 2-4 assume `release/1.00` has been merged to `main` — first raised on 12 September, still not done, and it blocks the entire CI workstream. Days 5-7 assume the Foundry provider is configured. **Neither is a QA task**, and both sit on the critical path. If either slips past day 5, the corresponding exit criteria will not be met and that should be an explicit, recorded decision rather than a surprise on day 13.

Excludes unit testing, which the development team owns. All current test counts and defect references are drawn from executions against a running local stack on 12 September 2026 and a code review of the increment to 17 September. Companion documents: *QA & BA Strategy*, *QA Test Case Register*, *Deployment Increment Review*, *User Story Revision Draft*.